

Chapter 37: Union-Find (DSU)

Personal Notes

Some problems come down to a deceptively simple question: are two things in the same group? Are these two people friends-of-friends? Are these two cells in the same island? Are these two nodes connected? For thousands of edges and millions of queries, running DFS every time would be far too slow.

Union-Find (also called Disjoint Set Union, or DSU) is a specialized data structure that answers these questions in essentially constant time. It's tiny — just two operations, about 20 lines of code — but the ideas behind it are ridiculously elegant. Once you learn it, you'll spot dozens of problems where it turns a hard algorithm into a mechanical one.

1. What Problem Does It Solve?

Union-Find manages a collection of DISJOINT SETS (groups that share no elements). It supports two operations, both in near-constant time:

- `find(x)` — which group does `x` belong to?
- `union(x, y)` — merge `x`'s group with `y`'s group

Initially: {1} {2} {3} {4} {5} (5 separate groups)

`union(1, 2):` {1, 2} {3} {4} {5}

`union(3, 4):` {1, 2} {3, 4} {5}

`union(1, 3):` {1, 2, 3, 4} {5}

`find(2) → same group as find(4)?` TRUE (both in {1,2,3,4})

`find(1) → same group as find(5)?` FALSE

Mental model: Think of it as a lightning-fast "are we cousins?" tester. You keep merging groups with `union`, and at any moment you can ask "are these two things related?" and get an answer in essentially $O(1)$ time. That's dramatically faster than searching a graph every time.

2. The Core Idea — Parent Pointers

Each element points to a PARENT in its group. Following parent pointers upward eventually leads to a ROOT — the representative of the group. Two elements are in the same group if and only if they have the same root.

Elements 0-6, initially each is its own group:

0 1 2 3 4 5 6

$\uparrow$ $\uparrow$ $\uparrow$ $\uparrow$ $\uparrow$ $\uparrow$ $\uparrow$
 0 1 2 3 4 5 6 $\leftarrow \text{parent}[i] = i$

After union(0,1), union(2,3), union(0,2):



parent = [0, 0, 0, 2, 4, 5, 6]

find(3): 3 → 2 → 0 (root). Group is 0.

find(1): 1 → 0 (root). Group is 0. Same group ✓

The trick is: store parents in a simple array, walk up by following parent[x] until you hit an element that's its own parent (the root).

3. Basic Implementation

The Class

```

class UnionFind {
    int[] parent;

    public UnionFind(int n) {
        parent = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;    // each is its own
parent
    }

    public int find(int x) {
        while (parent[x] != x) x = parent[x];
        return x;
    }

    public void union(int a, int b) {
        int rootA = find(a);
        int rootB = find(b);
        if (rootA != rootB) parent[rootA] = rootB;    // link one root
under the other
    }

    public boolean connected(int a, int b) {
        return find(a) == find(b);
    }
}
  
```

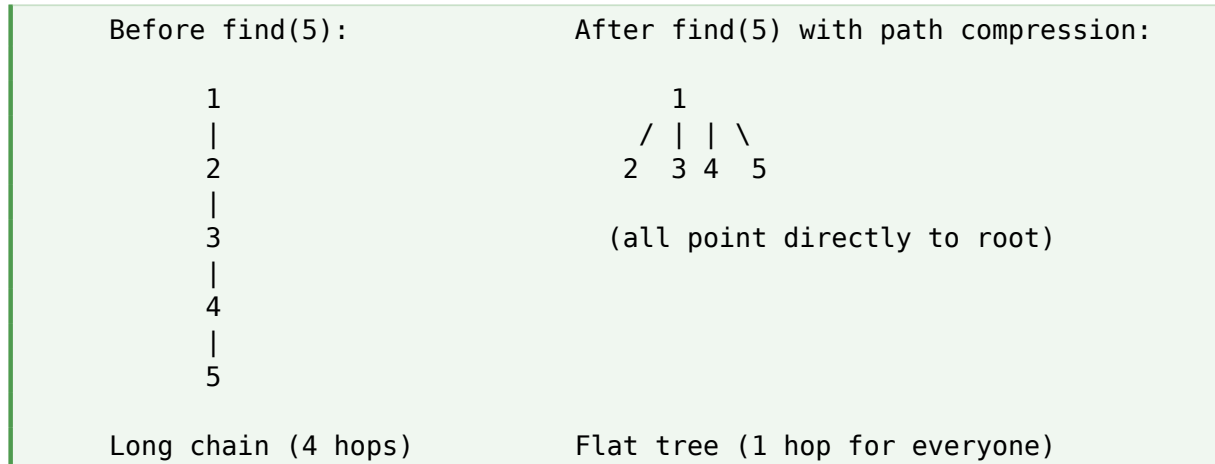
How the Operations Work

- Constructor — every element starts alone as its own root
- find — walk up parent pointers until you find a self-parent (the root)
- union — find both roots, then point one root at the other
- connected — two elements are in the same group iff they share a root

Basic version can degrade: Without optimizations, repeated unions can produce a LONG CHAIN — like $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$. Then $\text{find}(5)$ is $O(n)$. To keep operations near- $O(1)$, we need two optimizations coming next: path compression and union by rank.

4. Optimization 1 — Path Compression

Every time you call $\text{find}(x)$ and walk up to the root, you can FLATTEN the path — make every node on the way point DIRECTLY to the root. Next time, find is $O(1)$.



Recursive Version (cleanest)

```
public int find(int x) {
    if (parent[x] != x) {
        parent[x] = find(parent[x]);    // recurse and flatten
    }
    return parent[x];
}
```

Iterative Version

```
public int find(int x) {
    int root = x;
    while (parent[root] != root) root = parent[root];

    // Second pass: point every node on the path to the root
    while (parent[x] != root) {
        int next = parent[x];
```

```

        parent[x] = root;
        x = next;
    }
    return root;
}

```

5. Optimization 2 — Union by Rank (or Size)

When merging two groups, we can make the SHORTER tree hang under the TALLER one. This keeps the tree balanced — never becomes a tall skinny chain.

Union by Rank

```

class UnionFind {
    int[] parent;
    int[] rank;           // approximate height of each tree

    public UnionFind(int n) {
        parent = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
    }

    public int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }

    public void union(int a, int b) {
        int rootA = find(a), rootB = find(b);
        if (rootA == rootB) return;

        // Attach smaller tree under larger tree
        if (rank[rootA] < rank[rootB]) {
            parent[rootA] = rootB;
        } else if (rank[rootA] > rank[rootB]) {
            parent[rootB] = rootA;
        } else {
            parent[rootB] = rootA;
            rank[rootA]++; // only equal-height merge grows the rank
        }
    }
}

```

Union by Size — Alternative

```

int[] size; // size of the tree rooted at each node

// In union:

```

```

if (size[rootA] < size[rootB]) {
    parent[rootA] = rootB;
    size[rootB] += size[rootA];
} else {
    parent[rootB] = rootA;
    size[rootA] += size[rootB];
}

```

Rank vs Size: Both work well. Union by RANK tracks tree HEIGHT (a shorter tree hangs under a taller one). Union by SIZE tracks tree ELEMENT COUNT (a smaller group hangs under a bigger one). Size is often more useful because you can also answer "how big is this group?" for free. Pick one and stick with it.

6. The Complete Optimized DSU

Path compression + union by size — this is the version to memorize for interviews.

```

class UnionFind {
    int[] parent, size;
    int components;    // number of separate groups remaining

    public UnionFind(int n) {
        parent = new int[n];
        size = new int[n];
        components = n;
        for (int i = 0; i < n; i++) {
            parent[i] = i;
            size[i] = 1;
        }
    }

    public int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }

    public boolean union(int a, int b) {
        int rootA = find(a), rootB = find(b);
        if (rootA == rootB) return false;

        if (size[rootA] < size[rootB]) {
            int tmp = rootA; rootA = rootB; rootB = tmp;
        }
        parent[rootB] = rootA;
        size[rootA] += size[rootB];
        components--;
        return true;
    }
}

```

```

    public boolean connected(int a, int b) {
        return find(a) == find(b);
    }

    public int getSize(int x) {
        return size[find(x)];
    }

    public int count() {
        return components;
    }
}

```

The bonus features: components tracks how many separate groups remain — great for "count connected components" problems. getSize(x) tells you how big a group is. union returns true if a merge actually happened, useful for MST problems (where you only pay for edges that connect NEW groups).

7. Time Complexity

Operation	Basic (no opts)	With opts
find (worst case)	$O(n)$ — long chain	$O(\alpha(n)) \approx O(1)$
union (worst case)	$O(n)$ — dominated by find	$O(\alpha(n)) \approx O(1)$
connected	$O(n)$	$O(\alpha(n)) \approx O(1)$
Total for m operations	$O(m \times n)$	$O(m \times \alpha(n))$

$\alpha(n)$ is the INVERSE ACKERMANN function — one of the slowest-growing functions in mathematics. For every n small enough to fit in the observable universe, $\alpha(n) \leq 4$. That's why we say Union-Find is effectively $O(1)$ per operation.

For interviews: You can say Union-Find operations are "nearly constant time," or " $O(\alpha(n))$ amortized which is essentially $O(1)$," and be technically correct. Don't stress the alpha notation — the practical takeaway is: incredibly fast.

8. Classic Problem 1 — Number of Connected Components

Given n nodes and a list of edges, count how many separate groups exist. This is the textbook DSU problem.

```

public int countComponents(int n, int[][] edges) {
    UnionFind uf = new UnionFind(n);
}

```

```

    for (int[] edge : edges) {
        uf.union(edge[0], edge[1]);
    }
    return uf.count();
}

```

Dry Run

`n = 5, edges = [[0,1], [1,2], [3,4]]`

Initially: {0} {1} {2} {3} {4} components = 5

union(0, 1) → {0, 1} {2} {3} {4} components = 4

union(1, 2) → {0, 1, 2} {3} {4} components = 3

union(3, 4) → {0, 1, 2} {3, 4} components = 2

Answer: 2

9. Classic Problem 2 — Redundant Connection

You're given N edges of what should be a TREE (N nodes, $N-1$ edges), but one extra edge was added creating a cycle. Return the extra edge.

The Insight

Process edges one by one. For each edge (a, b), if a and b are ALREADY connected, this edge creates a cycle — return it. Otherwise, union them and continue.

```

public int[] findRedundantConnection(int[][] edges) {
    UnionFind uf = new UnionFind(edges.length + 1);
    for (int[] edge : edges) {
        if (!uf.union(edge[0], edge[1])) {
            return edge;    // union returned false → already connected →
            cycle!
        }
    }
    return new int[0];
}

```

The pattern: Any problem where you need to detect a cycle while incrementally adding edges is a perfect fit for DSU. The moment a union would join two ALREADY-connected components, you've found the cycle.

10. Classic Problem 3 — Number of Islands (DSU Style)

You've solved this with DFS in Chapter 33. Here's the DSU variant — the two-dimensional grid becomes a graph, each land cell (a '1') is a node, and adjacent land cells are edges.

```

public int numIslands(char[][] grid) {
    int rows = grid.length, cols = grid[0].length;
    UnionFind uf = new UnionFind(rows * cols);
    int water = 0;

    int[][] dirs = {{1, 0}, {0, 1}}; // down, right (avoid double-
    counting)

    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            if (grid[r][c] == '0') { water++; continue; }

            for (int[] d : dirs) {
                int nr = r + d[0], nc = c + d[1];
                if (nr < rows && nc < cols && grid[nr][nc] == '1') {
                    uf.union(r * cols + c, nr * cols + nc);
                }
            }
        }
    }

    return uf.count() - water;
}

```

The trick: flatten the 2D grid into 1D indices using $(r * \text{cols} + c)$. Then it's a regular graph.

11. Classic Problem 4 — Accounts Merge

Given a list of accounts where each has a name and multiple emails, merge accounts that share any email. Return one merged entry per person.

The Approach

- Assign each account an index (0 to n-1)
- For every email, remember which account first "owned" it
- When an email appears in a new account, union its owner with the previous owner
- After processing all accounts, group emails by their root

```

public List<List<String>> accountsMerge(List<List<String>> accounts) {
    int n = accounts.size();
    UnionFind uf = new UnionFind(n);
    Map<String, Integer> emailToAccount = new HashMap<>();

    // Union accounts that share an email
    for (int i = 0; i < n; i++) {
        for (int j = 1; j < accounts.get(i).size(); j++) {
            String email = accounts.get(i).get(j);
            if (emailToAccount.containsKey(email)) {
                uf.union(i, emailToAccount.get(email));
            }
        }
    }
}

```

```

        } else {
            emailToAccount.put(email, i);
        }
    }
}

// Group emails by their root
Map<Integer, TreeSet<String>> merged = new HashMap<>();
for (var e : emailToAccount.entrySet()) {
    int root = uf.find(e.getValue());
    merged.computeIfAbsent(root, k -> new
TreeSet<>()).add(e.getKey());
}

// Build result
List<List<String>> result = new ArrayList<>();
for (var e : merged.entrySet()) {
    List<String> account = new ArrayList<>();
    account.add(accounts.get(e.getKey()).get(0)); // name
    account.addAll(e.getValue()); // sorted emails
    result.add(account);
}
return result;
}

```

12. Classic Problem 5 — Kruskal's Minimum Spanning Tree

A CLASSIC use case for DSU. Kruskal's algorithm builds an MST by processing edges in weight order, adding each one if it doesn't create a cycle. DSU makes the cycle check $O(\alpha(n))$.

Algorithm

1. Sort all edges by weight (ascending)
2. For each edge in order: if the two endpoints are NOT already connected, add the edge to the MST and union them
3. Stop when you have $n-1$ edges (a spanning tree of n nodes)

```

public int minSpanningTree(int n, int[][] edges) {
    // edges[i] = [u, v, weight]
    Arrays.sort(edges, (a, b) -> a[2] - b[2]);

    UnionFind uf = new UnionFind(n);
    int totalCost = 0, edgesUsed = 0;

    for (int[] edge : edges) {
        if (uf.union(edge[0], edge[1])) {
            totalCost += edge[2];
            edgesUsed++;
            if (edgesUsed == n - 1) break;
        }
    }
}

```

```

    }
}

return edgesUsed == n - 1 ? totalCost : -1;    // -1 = disconnected
}

// Time: O(E log E) for sort, plus O(E × α(V)) for unions.

```

Why DSU is perfect here: Kruskal needs to answer "would this edge create a cycle?" for every edge. Without DSU, you'd have to run DFS or BFS each time — turning the algorithm from $O(E \log E)$ into something much slower. DSU answers the question in $O(\alpha(V))$ per edge, which is essentially free.

13. When to Reach for Union-Find

If the problem asks...	Use
Count connected components	DSU — the classic use case
Detect cycles while adding edges	Union each edge; if already connected → cycle
Find MST (Kruskal's algorithm)	Sort edges + DSU cycle check
Merge accounts / groups sharing an attribute	Union accounts that share the attribute
Number of Islands (2D grid version)	Flatten grid to 1D, union adjacent land cells
Friend circles / social groups	Union any two people who are connected
Percolation / connectivity queries	Batch queries after all unions are done
Range-related grouping with equivalence	DSU on ranges or equivalence classes

14. DSU vs BFS/DFS for Connectivity

Scenario	Union-Find	BFS / DFS
Static graph	Both work well	Use whichever is simpler
Edges arrive online	Great — union each new edge	Costly — rebuild each time
Only need to KNOW if connected	Perfect — no path needed	Overkill
Need the actual PATH	Not designed for this	Use DFS/BFS instead
Detect cycles incrementally	Perfect fit	Slower

Scenario	Union-Find	BFS / DFS
Component count	count is a natural byproduct	Requires post-processing

The tie-breaker: if edges arrive one at a time and you need to keep answering "are these connected?", DSU wins by a mile. For static "trace this graph" problems, DFS/BFS are usually more natural.

15. Common Mistakes

Mistake 1: Skipping path compression

Without path compression, find can degrade to $O(n)$ for long chains. Always include it — it's just one line of extra code and gives you the near- $O(1)$ magic.

Mistake 2: Comparing values directly instead of roots

```
if (a == b) return;           // x WRONG – checks values, not group
membership
if (find(a) == find(b)) ...  // ✓ correct – checks group
```

Mistake 3: Forgetting to initialize parent[i] = i

Without this, parent[i] defaults to 0 (in Java int[]), and every element would appear to be in group 0. Loop through and set parent[i] = i in the constructor.

Mistake 4: Uniting under the wrong root

```
// BAD – this doesn't merge the whole groups
parent[a] = b;           // x only points a to b, other members of a's
group are still stranded

// GOOD – link ROOTS, not the elements themselves
parent[find(a)] = find(b); // ✓
```

Mistake 5: Off-by-one in sizes

If your nodes are labeled 1 to n (not 0 to n-1), remember to allocate the parent array of size n+1. Otherwise index n is out of bounds.

Mistake 6: Not tracking component count

For "count connected components" problems, add a counter that DECREASES on every successful union. Otherwise you'll have to walk the parent array afterward and count unique roots — slower and easier to get wrong.

Mistake 7: Using DSU for problems that need paths

DSU tells you IF two nodes are connected, not HOW they're connected. If you need to reconstruct the actual path, use BFS/DFS.

16. Best Practices

- Always include BOTH optimizations — path compression AND union by size/rank
- Prefer union by SIZE over rank — gives you group sizes for free
- Have union() return true/false — makes cycle detection a one-liner
- Keep a components counter that decrements on every successful union
- For 2D grid problems, flatten indices as $r * \text{cols} + c$
- When labels are strings (like account emails), map them to integers first
- Memorize the ~20-line optimized DSU — it's a common interview building block
- Reach for DSU whenever edges arrive online or you need repeated connectivity queries

17. Quick Reference

Concept	Meaning
Disjoint Set Union (DSU)	Data structure for managing groups
Union-Find	Another name for DSU
find(x)	Which group does x belong to? Returns root.
union(a, b)	Merge groups containing a and b
connected(a, b)	Are a and b in the same group? <code>find(a) == find(b)</code>
parent[]	Array where <code>parent[i]</code> is i's direct parent
Root	An element whose parent is itself
Path compression	Flatten <code>find()</code> paths to make everyone point at the root
Union by rank	Attach shorter tree under taller tree
Union by size	Attach smaller group under larger group
$\alpha(n)$	Inverse Ackermann — essentially constant
components	Count of separate groups remaining

18. Practice Problems

Try in order — the first four are must-know for interviews.

Foundational

4. Implement Union-Find from scratch with path compression and union by size.
5. Number of Connected Components in an Undirected Graph (LeetCode 323).
6. Number of Provinces (LeetCode 547) — friend circles with an adjacency matrix.
7. Graph Valid Tree — is the given graph a tree? (LeetCode 261).

Cycle Detection

8. Redundant Connection (LeetCode 684).
9. Redundant Connection II (LeetCode 685) — for directed graphs (harder).
10. Detect cycle in an undirected graph using Union-Find.

Grid Problems

11. Number of Islands using DSU (LeetCode 200 — DSU variant).
12. Number of Islands II (LeetCode 305) — islands added one at a time online.
13. Surrounded Regions using DSU (LeetCode 130).
14. Making a Large Island (LeetCode 827) — union all cells then try flipping each 0.

Advanced

15. Accounts Merge (LeetCode 721).
16. Most Stones Removed with Same Row or Column (LeetCode 947).
17. Satisfiability of Equality Equations (LeetCode 990).
18. Kruskal's Minimum Spanning Tree from an edge list.
19. Connecting Cities With Minimum Cost (LeetCode 1135) — MST application.
20. Number of Operations to Make Network Connected (LeetCode 1319).
21. Similar String Groups (LeetCode 839).

19. Final Takeaway

Union-Find is a small tool with a large impact. The full implementation is only ~20 lines, but it makes previously $O(n^2)$ connectivity problems feel almost free. Once you can implement the optimized version from memory and recognize the "are these connected?" or "count groups" pattern, an entire category of interview problems becomes routine.

For problems where edges arrive one at a time — friend requests, network connections, incrementally revealed data — DSU is often the ONLY practical choice. It's the specialized tool that beats generic graph traversal cleanly.

Key Takeaway: DSU manages disjoint groups with two operations: `find(x)` and `union(a, b)`. Use path compression + union by size for near- $O(1)$ operations. Track components as unions happen. Perfect

for connectivity queries, cycle detection, Kruskal's MST, and grid grouping. Memorize the ~20-line optimized version — it's one of the most reusable pieces of code you'll ever write.

20. What's Next?

With DSU under your belt, the remaining DSA specializations are:

- Bit Manipulation — subset generation with bitmasks, XOR tricks
- Monotonic Stack / Deque — next greater element, largest rectangle
- Two Pointers in depth — palindrome, 3-sum, container with most water
- Prefix Sums — subarray sums with negatives and range queries
- Segment Tree / Fenwick Tree — range queries with updates
- Advanced Strings — KMP, Rabin-Karp for pattern matching
- Advanced Graphs — Bellman-Ford, Floyd-Warshall, articulation points
- System Design — the next big step beyond DSA